

IZS

TERAMO

/

ISTITUTO
ZOOPROFILATTICO
SPERIMENTALE
DELL'ABRUZZO
E DEL MOLISE
"G. CAPORALE"

From epidemiological question to executable pipeline

LLM-assisted Nextflow
DSL2 synthesis, on-premise
a proof of concept

Andrea de Ruvo

Istituto Zooprofilattico Sperimentale
dell'Abruzzo e del Molise "G. Caporale"

GMI16 – 25 September 2026 – Tunis



Acknowledgements

Developed as a thesis research project by three students of the **EDISS** programme, in collaboration with **IZS Teramo**:

Martinus Grady	lead architecture & agent engineering
Ligan Cai	co-author, evaluation
Zeynal Mardanli	co-author, frontend



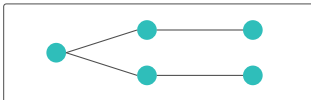
What it does

In one slide, before the rest of the talk

a question, in plain English

"I have Illumina paired-end reads of Salmonella isolates. I want to do SNP-based cluster analysis across samples. What do you suggest?"

or a canvas



Cohesive LLM

one GPU
≈ 50 s

 **cohesive-ngsmanager**

52 components
+ knowledge graph

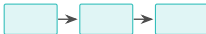
read at query time,
not trained into the model

 **genpat-it/**
cohesive-ngsmanager

an executable pipeline

pipeline.nf

the topology

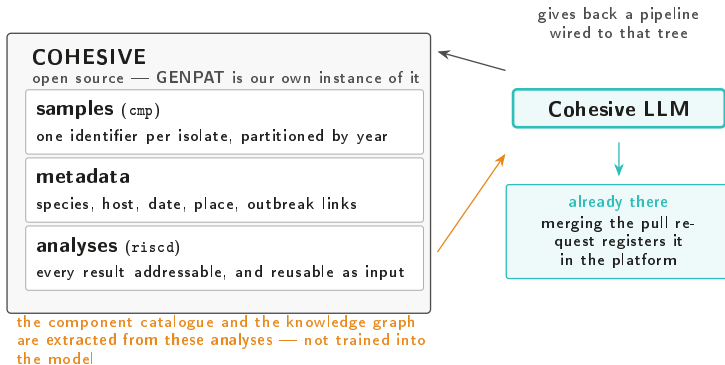


checked by Nextflow

what is missing to run

Where the pipelines live: COHESIVE

The open-source release of GENPAT, the instance IZS Teramo runs in house



IZS

T E R A M O

/

ISTITUTO
ZOOPROFILATTICO
SPERIMENTALE
DELL'ABRUZZO
E DEL MOLISE
"G. CAPORALE"

Demo

from question to validated pipeline

Three recordings — the whole loop 6:12, the canvas 1:26, the results
1:01

 watch, low-res

 watch, high-res

Who it is for

Two kinds of user, and a picture both can read

1. Domain experts are demanding, but not coders.

An epidemiologist knows exactly which analysis is needed and is exacting about its scientific correctness — but cannot write DSL2, and today waits in the bioinformatics queue.

2. Experts should not start from a blank file either.

A generated skeleton, grounded in the existing catalogue and already validated, is a **starting point to edit** — not a replacement for expertise.

3. A diagram is read faster than code.

Every pipeline ships with a Mermaid flowchart: the expert checks the topology at a glance, the non-coder sees what will actually run.

Same tool, two entry points: a question in plain English, or a canvas.

Conclusions

What I would like you to take away

- **It supports the curator, it does not replace one.** The agent proposes; a person merges.
- **Constrained by design** — it composes only components already running in production, which is what makes reproducibility possible.
- **Local, and not only for the EU AI Act.** One on-premise GPU: nothing leaves the institute. The next step is to let it read the **sample metadata** as well — and that is data we cannot send anywhere. Guardrails are also easier when the output space is this narrow.
- **Portable** — nothing here is specific to GENPAT: point it at another catalogue and it works.

IZS

TERAMO

ISTITUTO
ZOOPROFILATTICO
SPERIMENTALE
DELL'ABRUZZO
E DEL MOLISE
"G. CAPORALE"

Thank you

Questions?

Andrea de Ruvo — a.deruvo@izs.it

 [genpat-it/cohesive-llm](https://github.com/genpat-it/cohesive-llm)

Technical slides follow, for anyone who wants the detail.

The hardware, and its ceiling

One card, and why a bigger model would not help

GPU	NVIDIA L40S , 48 GB, PCIe 4.0 ×16
Model	Qwen3.8-27B, FP8 — ≈ 27 GB of weights
Server	vLLM, prefix caching, speculative decoding
Context	65 536 tokens, of which 24 576 reserved for the answer

Single-sequence decoding is bound by memory bandwidth, not by compute:

$864 \text{ GB/s} \div 27 \text{ GB} \approx \mathbf{32 \text{ tokens/s}}$ in theory — **44 tokens/s** measured

We are *above* the theoretical ceiling, thanks to speculative decoding.

Tuning the engine further gained 5–8 %; the real gain came from elsewhere — see the next slide.

Determinism, measured

The same request, twelve times

A generative model gives a different answer every time. For a demo — and for a laboratory — that is the property that matters, so we measured it rather than hoped.

	before	after
identical pipelines, same prompt	1 in 4 runs	12 in 12
a step that fails on <i>Listeria</i>	appeared	never
mean time per turn	51.7 s	38.6 s

What changed was **not** the model and **not** the prompt: three entries in the component catalogue were wrong. One tool that screens virulence was described as a resistance tool; the keywords written for search were never indexed; one step's input shape was undeclared.

Better retrieval shortens the reasoning loop. That is where the 13 seconds
from the CPU

Why not a general coding agent

Claude Code, Codex, and why not yet

- **Open output space.** They can invent a process, call a tool we do not have, or write something that runs but is not what the laboratory validated. Here the model emits a typed AST restricted to components already in production.
- **Guardrails and sandboxing are easier** when the output space is this narrow: there is a finite list of things the model is allowed to say.
- **The data.** The next step is to let the agent read sample metadata. That cannot leave the institute — so the inference cannot either.
- **Not a closed door.** A hosted API can be plugged in where confidentiality allows it: the architecture does not depend on which model answers.

What is missing

Honest roadmap

- **No automatic correction loop.** When validation fails, the error goes back to the model *by hand* and it fixes itself. It should close that loop on its own.
- **Metadata and data.** Connect the agent to the platform through MCP, so it can reason over samples and results, not only over pipelines.
- **Learning from use.** Curators approve, edit or reject every proposal — a signal we currently throw away.
- **From a form to a conversation.** The interface is still a chat window bolted onto a tool; it should feel like talking to the platform.

The bottleneck is not the sequencing

During an outbreak:

- the epidemiological question is there **immediately**
- sequencing data arrives in days
- the **pipeline** arrives when a bioinformatician frees up

Writing Nextflow DSL2 by hand requires:

- knowing the component catalogue and the exact channel names
- synthesising the right operators (`.cross()`, `.multiMap{}`, `.combine()`)
- iterating by trial and error

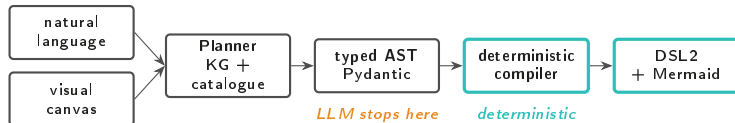
The scarce resource in outbreak response is not the GPU. It is bioinformatician time.

An LLM told to “write the pipeline”:

- invents tools that are not in the framework
- invents plausible channel names
- produces code that *looks* right
- needs brittle error-repair loops

Plausible-looking Nextflow is worse than no Nextflow: it costs a bioinformatician a debugging session to discover it is wrong.

The LLM is the architect, not the programmer



- The LLM emits a **typed AST**, not a file of code.
- A deterministic compiler assembles the pipeline around it: imports, workflow interfaces and entrypoint are never model-authored text.
- **No error-repair loop.**

What is an AST?

And why the model emits one instead of code

An **abstract syntax tree** is the structure of a program before anyone writes it out as text: which pieces there are, and how they are wired. A compiler turns that structure into source code — always the same way.



Why it matters: a **tree can be checked** — does this component exist, does it emit that channel, is this import allowed. Free text cannot. The model is never in a position to invent a process that does not exist, because there is nowhere in the tree to put one.

What the model actually emits

A real AST, trimmed

```
{ "imports": [
  { "module_path": "../functions/parameters.nf",
    "functions": ["getSingleInput"] },
  { "module_path": "../steps/step_1PP_trimming__fastp",
    "functions": ["step_1PP_trimming__fastp"] }, ... 2 more
],
  "sub_workflows": [
    { "name": "ASSEMBLY_AND_PREPROCESSING_AND_TX",
      "take_channels": ["rawreads"],
      "emit_channels": ["trimmed = trimmed", "assembled = assembled"],
      "body_code": "trimmed = step_1PP_trimming__fastp(rawreads).trimmed
                    assembled = step_2AS_denovo__spades(trimmed).assembled
                    report = step_3TX_class__kraken2(trimmed).genus_report"
    }
  ],
  "entrypoint": { "body_code": "rawreads = getSingleInput()
                               ASSEMBLY_AND_PREPROCESSING_AND_TX(rawreads)" }}
```

Typed and validated: imports, sub-workflow interface, take/emit channels, void-tool rules.

Still a string: body_code — the dataflow statements. Regex-checked, not structural.

How the model finds a component

Vector search, and why it decides the outcome

The catalogue holds 52 components. The model never sees all of them: it **searches**, and only what the search returns can end up in a pipeline.

Embedding	each component becomes a vector, built from its id, tool, domain, description and keywords
FAISS	the index that finds the nearest vectors to a query, in milliseconds, on the CPU, with no network
Query	the scientific intent, not a tool name: <i>"screen its virulence gene content"</i>

A component described badly cannot be found, however good the model is.

Ours called a virulence screen a resistance tool, and indexed everything *except* the keywords written for searching. Fixing the catalogue, not the model, is what made the same request deterministic.

The planner queries a knowledge graph built from **cohesive-ngsmanager**, the framework in production at IZS.

Tier	Source	Confidence
EXTRACTED	real AST wiring	verified
INFERRED	template co-occurrence	probable
AMBIGUOUS	channel-name matching	heuristic

- The graph separates **what it has seen** from what it is guessing.
- The reasoning core holds **zero** tool names: catalogue, channels and rules all come from a plugin.

One server, inside the institute

- Qwen3.8-27B-FP8 on **a single** NVIDIA L40S (46 GB)
- vLLM, 65k context, FP8 KV cache, under 44 GB VRAM
- no data leaves the network

Measured on the hardware

generation throughput	36 tok/s (31–45)
time to first token	0.15–0.22 s
pipeline from text	~50 s
pipeline from canvas	~20 s

Measured, not estimated.

I am not asking you to trust the model

The generated code is handed to **real Nextflow 22.10**, in -preview, against the production framework.

Pipeline generated from natural language:

```
success      true
warnings     missing required param:
              multi_clustering__reporttree__summary_columns
```

Negative control — non-existent component:

```
success      false
errors       Can't find a matching module file
              for include: ../steps/step_DOES_NOT_EXIST
```

The includes resolve, the syntax holds, and validation names exactly which parameter is missing.

Where it stops

Stated up front, before you find it

1. Validation stops at the imports.

The framework's parameter gate exits before Nextflow resolves the workflow body, so a call to a process that was never included still passes as success: `true`.

2. Dataflow topology can be wrong.

In one pipeline the clustering step received the trimmed reads instead of the VCFs from the preceding step.

3. The catalogue constraint is not enforced at render time.

Process calls inside `body_code` are never reconciled against the `imports` list, so a call can appear with no `include`. First on the roadmap.

An accelerator, not a replacement. Human review stays in the loop.

Your framework, not ours

The reasoning engine is **domain-agnostic**: no tool names, no organisms, no domain-specific rules in the core. A new institute adds a **plugin** — component catalogue, channels, pipeline templates.

Current state

IZS components	52
registry	70 components, 30 void tools
offline test suite	33/33 passing
licence	MIT

`github.com/genpat-it/cohesive-llm`

A new plugin, not a fork.

Demo video

Everything shown here, and the code behind it

Recording	low-res	high-res
The whole loop, 6:12	 46 MB	Drive, 418 MB
The canvas, 1:26	 9 MB	Drive, 74 MB
The results, 1:01	 13 MB	Drive, 75 MB

The repositories involved, at the revision that produced the demo:

Repository	Branch	
 genpat-it/cohesive-llm	gmi16	the agent
 genpat-it/cohesive-ngsmanager	main	the framework

The high-resolution files keep the original 2734×1536 at 30 fps; the low-resolution ones are the same recordings re-encoded.

In summary

1. The LLM **does not write code**: it emits a typed AST, and a deterministic compiler generates the DSL2.
2. Reasoning is **grounded** in a graph extracted from pipelines running in production, with confidence tiers made explicit.
3. The generated code is **validated by real Nextflow** against the production framework.
4. Everything runs on **one on-premise GPU**: no isolate data leaves the institute.

We are looking for laboratories willing to try it on their own catalogue.

Andrea de Ruvo — a.deruvo@izs.it

Component	Role
Planner subgraph	ReAct consultant, graph traversal, extractor
Execution subgraph	void-tool precheck, AST generation, renderer
Orchestration	hierarchical LangGraph state machine
Memory	lossless compaction of tool facts
Retrieval	FAISS + patterns index
Model	Qwen3.8-27B-FP8, vLLM, MTP speculative $k=3$

Backup — an example of the output

```
include { getSingleInput } from '../functions/parameters.nf'
include { step_3TX_class__kraken2 } from '../steps/step_3TX_class__kraken2'
include { step_1PP_trimming__fastp } from '../steps/step_1PP_trimming__fastp'
include { step_2AS_denovo__spades } from '../steps/step_2AS_denovo__spades'

workflow ASSEMBLY_AND_PREPROCESSING_AND_TX {
  take:
    rawreads
  main:
    trimmed = step_1PP_trimming__fastp(rawreads).trimmed
    assembled = step_2AS_denovo__spades(trimmed).assembled
    report = step_3TX_class__kraken2(trimmed).genus_report
  emit:
    trimmed = trimmed
    assembled = assembled
}

workflow {
  rawreads = getSingleInput()
  ASSEMBLY_AND_PREPROCESSING_AND_TX(rawreads)
}
```

Generated from the canvas in ~20 s. Validated success: true, warning
missing required param: step_3TX_class__kraken2__db.